

GUÍA PARA DESARROLLAR EN PLANTUML DIAGRAMAS DE CASOS DE USO DE ANÁLISIS

Ejemplo base: Finca Jirah

Asignatura: Análisis y Diseño de Software

Tema: Modelado de casos de uso de nivel 0 y apoyo al análisis orientado a objetos con PlantUML

Caso base: Gestión de la finca Jirah con los roles Administrador y Agricultor

Propósito: orientar al estudiante para pasar desde requisitos funcionales de nivel 0 hacia casos de uso claros, trazables y verificables.

1. Propósito de la guía

Esta guía orienta el desarrollo de diagramas de casos de uso de análisis en PlantUML, tomando como referencia una secuencia académica progresiva: partir de una ERS/SRS, seleccionar requisitos funcionales de nivel 0, identificar actores, modelar casos de uso principales y mantener trazabilidad.

El ejemplo se centra en el Sistema de Gestión Agrícola (SGA). Para efectos de análisis, se abordarán las funcionalidades principales que permiten a los perfiles de Administrador y Agricultor interactuar con el sistema.

2. Relación con la ruta de análisis de la Unidad 1

Momento	Pregunta guía	Producto esperado	Relación con la siguiente tarea
U1T1 - Análisis funcional inicial	¿Qué funcionalidades de nivel 0 requiere el sistema y qué actores participan?	Diagrama de casos de uso en PlantUML y matriz RF-CU .	Permite identificar clases, atributos y responsabilidades para el análisis estructural .
U1T2 - Modelado estructural	¿Qué clases principales se derivan de los casos de uso?	Diagrama de clases de análisis con clases de entidad, control y frontera cuando corresponda.	Entrega objetos base para diagramas de secuencia.
U1T3 - Modelado dinámico	¿Cómo interactúan actores, objetos y clases para cumplir cada caso de uso?	Diagramas de secuencia derivados de los casos de uso y clases de análisis.	Cierra la trazabilidad requisito-caso-clase-secuencia.

3. Conceptos mínimos para modelar casos de uso

- **Actor:** Rol externo que interactúa con el sistema. Puede ser una persona, área, dispositivo u otro sistema.
- **Caso de uso:** **Funcionalidad observable desde el punto de vista del actor. Debe nombrarse con verbo + objeto, por ejemplo: Agregar estudiante.**
- **Límite del sistema:** Caja o frontera que delimita qué funcionalidades pertenecen al sistema y qué elementos están fuera.
- **Relación actor-caso de uso:** Comunicación directa entre un actor y una funcionalidad del sistema.
- **Relación <<include>>:** Se usa cuando un caso de uso siempre reutiliza un comportamiento obligatorio de otro.
- **Relación <<extend>>:** Se usa cuando un comportamiento opcional o condicionado amplía otro caso de uso.

Cuidado académico: Las relaciones <<include>> y <<extend>> no deben usarse para “decorar” el diagrama. **Solo se incorporan cuando aportan claridad al análisis y mantienen coherencia con los requisitos.**

4. Caso base: Sistema de Gestión Agrícola (SGA)

El sistema debe permitir gestionar una finca agrícola mediante dos perfiles principales: el Administrador de la Finca y el Agricultor. Las operaciones principales de nivel 0 disponibles son: Gestionar Agricultores, Monitorear Producción, Ingresar producción campo y Gestionar cuenta.

Dato	Tipo sugerido	Descripción	Validación mínima
Agricultor	Texto corto	Cuentas de usuarios gestionadas por el Administrador.	Nombre, cédula (única), correo y credenciales.Obligatorio y no repetido.
Registro de la producción recolectada en campo.	Double y entero	Registro de la producción recolectada en campo.	Peso de la gaveta, lote perteneciente.Obligatorio.
Rechazos	Texto	Registro de mermas o productos rechazados.	Obligatoria; Motivo del rechazo, lote y comentario.
Preferencias	Configuración de la interfaz del usuario.	Configuración de la interfaz del usuario.	Idioma y tema (claro /oscuro).

5. Requisitos funcionales de nivel 0 para el CRUD

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir crear, editar, eliminar y consultar las cuentas de los agricultores	Administrador	Gestionar Agricultores
RF-02	El sistema debe permitir visualizar métricas mediante un dashboard interactivo y generar reportes.	Administrador	Monitorear Producción
RF-03	El sistema debe permitir Ingresar producción, reportar rechazos y sincronizar datos de producción.	Agricultor	Ingresar producción campo
RF-04	El sistema debe permitir modificar la contraseña y modificar las preferencias del usuario actual.	Administrador, Agricultor	Gestionar cuenta
RF-05	El sistema debe validar los datos obligatorios antes de ingresar, reportar, guardar y sincronizar los datos.	Sistema	Validar datos de produccion, agricultores y administrador

6. Procedimiento para construir el diagrama en PlantUML

1. Revisar la descripción del problema SGA y seleccionar los 4 requisitos funcionales de nivel 0. Identificar actores primarios, secundarios y sistemas externos
2. Identificar actores primarios: Administrador y Agricultor.
3. Redactar casos de uso con nombres claros: Gestionar Agricultores, Monitorear Producción, Ingresar producción campo, Gestionar cuenta.
4. Dibujar el límite del sistema "Sistema Gestion Agricola (SGA)".

5. Agregar relaciones actor-caso de uso según los perfiles definidos en el documento.
6. Construir la matriz de trazabilidad RF-CU.
7. Validar el código PlantUML.

7. Código PlantUML: diagrama de casos de uso SGA

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

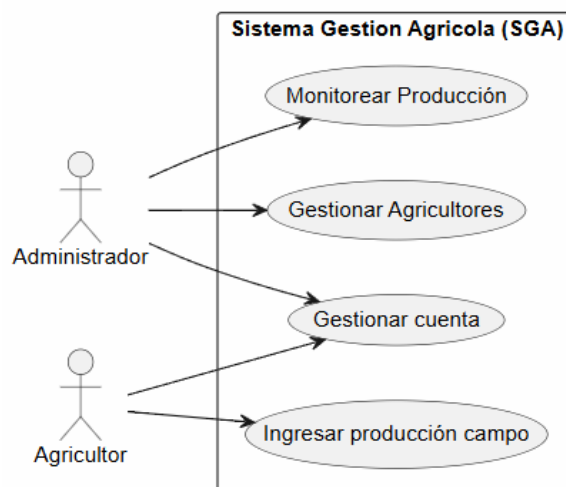
actor "Administrador" as Admin
actor "Agricultor" as Agric

rectangle "Sistema Gestion Agricola (SGA)" {
    usecase "Gestionar Agricultores" as UC1
    usecase "Monitorear Producción" as UC2
    usecase "Gestionar cuenta" as UC3
    usecase "Ingresar producción campo" as UC4
}

Admin --> UC1
Admin --> UC2
Admin --> UC3

Agric --> UC4
Agric --> UC3
```

Interpretación: El Administrador tiene acceso a la gestión de las cuentas de los agricultores, el monitoreo a través de dashboards y la gestión de su propia cuenta. El Agricultor, por su parte, se enfoca en ingresar la información recopilada en campo y gestionar sus preferencias de cuenta.



8. Especificación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Gestionar Agricultores	Administrador	El administrador ha iniciado sesión.	Ingresar nombre, cédula, correo -> validar -> guardar o actualizar información.	Cuentas de agricultores creadas/actualizadas correctamente.
Monitorear Producción	Administrador	El administrador ha iniciado sesión.	Seleccionar Dashboard -> aplicar filtros (fechas, agricultor, motivo) -> sistema actualiza.	Visualización de métricas.
Ingresar producción campo	Agricultor	Disponer de datos de recolección.	Ingresar peso de gaveta y lote (o motivo de rechazo) -> validar peso -> guardar o sincronizar.	Producción y rechazos almacenados.
Gestionar cuenta	Administrador, Agricultor	El usuario ha iniciado sesión.	Seleccionar preferencias/contraseña -> ingresar nuevos datos -> validar políticas -> actualizar.	Contraseña actualizada o preferencias almacenadas.

9. Puente hacia clases de análisis: frontera, control y entidad

Aunque esta guía se enfoca en casos de uso, los documentos base indican que el análisis orientado a objetos puede continuar **identificando clases de frontera, control y entidad**.

Tabla 1: Puente para UC1 - Gestionar Agricultores

Tipo de clase	Propósito en análisis	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa la pantalla de gestión del CRUD de agricultores.	boundary "VistaGestionAgricultores" as V_Agric
Control / Control	Coordina la lógica de crear, editar, eliminar y consultar.	control "ControladorAgricultores" as C_Agric
Entidad / Entity	Representa la información del agricultor.	entity "Agricultor" as E_Agric

Tabla 2: Puente para UC2 - Monitorear Producción

Tipo de clase	Propósito en análisis	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa el dashboard y la selección de filtros.	boundary "VistaMonitoreoProduccion" as V_Monit
Control / Control	Coordina la lógica de aplicar filtros y solicitar métricas.	control "ControladorMonitoreo" as C_Monit

Entidad / Entity	Representa los registros de producción a visualizar.	entity "Produccion" as E_Prod
------------------	--	-------------------------------

Tabla 3: Puente para UC3 - Gestionar Cuenta

Tipo de clase	Propósito en análisis	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa la pantalla de configuración de perfil y seguridad.	boundary "VistaGestionCuenta" as V_Cta
Control / Control	Coordina la actualización de contraseña y preferencias.	control "ControladorCuenta" as C_Cta
Entidad / Entity	Representa las credenciales y configuración del usuario.	entity "Cuenta" as E_Cta

Tabla 4: Puente para UC4 - Ingresar Producción Campo

Tipo de clase	Propósito en análisis	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa la pantalla para registrar en campo.	boundary "VistaIngresoProduccion" as V_Ingr
Control / Control	Coordina el registro de peso, lote, rechazos y sincronización.	control "ControladorProduccion" as C_Ingr
Entidad / Entity	Representa el dato recolectado en campo.	entity "Produccion" as E_Prod

Código complementario para visualizar el puente entre casos de uso y clases de análisis:

```
@startuml
left to right direction
skinparam shadowing false

actor "Administrador" as Admin
actor "Agricultor" as Agric

' VISTAS
boundary "VistaGestionAgricultores" as V_Agric
boundary "VistaMonitoreoProduccion" as V_Monit
```

```

boundary "VistaGestionCuenta" as V_Cta
boundary "VistaIngresoProduccion" as V_Ingr

' CONTROLADORES
control "ControladorAgricultores" as C_Agric
control "ControladorMonitoreo" as C_Monit
control "ControladorCuenta" as C_Cta
control "ControladorProduccion" as C_Ingr

' ENTIDADES
entity "Agricultor" as E_Agric
entity "Produccion" as E_Prod
entity "Cuenta" as E_Cta

' RELACIONES ADMIN
Admin --> V_Agric : gestiona (CRUD)
Admin --> V_Monit : filtra dashboard
Admin --> V_Cta : configura perfil

' RELACIONES AGRICULTOR
Agric --> V_Ingr : registra datos
Agric --> V_Cta : configura perfil

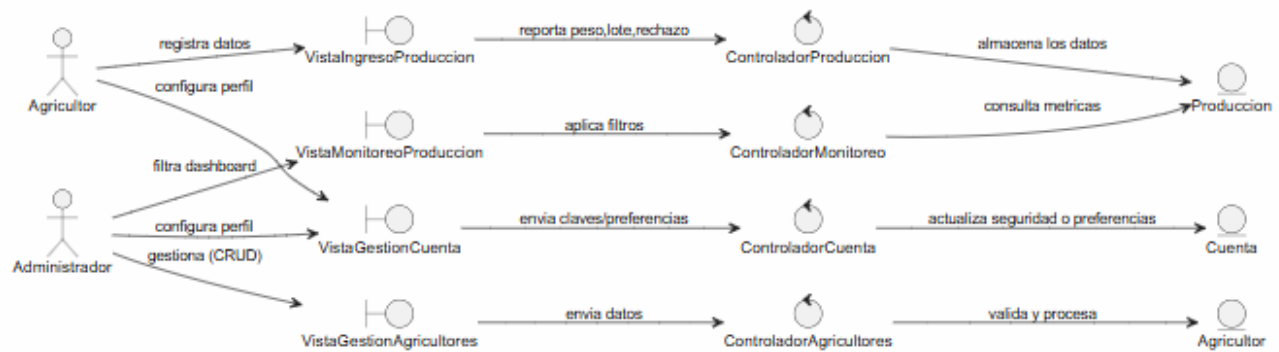
' FLUJO GESTION AGRICULTORES
V_Agric --> C_Agric : envia datos
C_Agric --> E_Agric : valida y procesa

' FLUJO MONITOREO
V_Monit --> C_Monit : aplica filtros
C_Monit --> E_Prod : consulta metricas

' FLUJO GESTION CUENTA
V_Cta --> C_Cta : envia claves/preferencias
C_Cta --> E_Cta : actualiza seguridad o preferencias

' FLUJO INGRESO PRODUCCION
V_Ingr --> C_Ingr : reporta peso,lote,rechazo
C_Ingr --> E_Prod : almacena los datos
@enduml

```



10. Matriz de trazabilidad RF-CU para el ejemplo

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Gestionar Agricultores	Administrador	Alta	El sistema permite crear, editar, consultar y eliminar validando que la cédula sea única.	UC1
RF-02	Monitorear Producción	Administrador	Media	El dashboard interactivo filtra y muestra gráficos de producción .	UC2
RF-03	Ingresar producción campo	Agricultor	Alta	El sistema guarda el peso y el lote, motivo de rechazo, permitiendo la sincronización de datos.	UC4
RF-04	Gestionar cuenta	Admin, Agricultor	Media	El sistema actualiza la contraseña según políticas de seguridad y guarda preferencias.	UC3

11. Lista de verificación para revisar el diagrama

- Cada caso de uso se deriva de un requisito funcional de nivel 0.
- El actor representa un rol externo, no una pantalla ni una clase interna.

- El nombre del caso de uso está redactado como objetivo del usuario.
- El límite del sistema está claramente identificado.
- Las relaciones <<include>> o <<extend>> tienen justificación funcional.
- El diagrama no modela detalles técnicos prematuros.
- El código PlantUML compila y genera un diagrama legible.
- La matriz RF-CU permite verificar trazabilidad sin pérdida de información.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad RF-CU	Relación clara entre requisitos funcionales, actor, caso de uso y criterio de aceptación.	5 pts	Matriz RF-CU completa.
Modelado de casos de uso	Representación correcta de actor, límite del sistema, casos de uso y relaciones.	5 pts	Diagrama PlantUML generado.
Claridad del análisis	Selección pertinente de funcionalidades de nivel 0 y nombres orientados a objetivos del usuario.	4 pts	Casos de uso redactados claramente.
Uso correcto de PlantUML	Código funcional, ordenado y comprensible.	3 pts	Código sin errores de sintaxis.
Presentación técnica	Documento ordenado, con evidencias, explicación breve y coherencia visual.	3 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Requisitos funcionales	Listado de RF de nivel 0 para el CRUD de Estudiante.	Tabla RF.
Diagrama de casos de uso	Código PlantUML y captura del diagrama generado.	Código + imagen.
Matriz de trazabilidad	Relación RF, actor, caso de uso, prioridad y criterio de aceptación.	Tabla RF-CU.
Conclusión técnica	Reflexión breve sobre trazabilidad, consistencia y utilidad del modelo.	Texto de cierre.

14. Fuentes internas consultadas

Documentos revisados desde la carpeta de Google Drive compartida por la docente:

- Modelado de Clases de Análisis en PlantUML.docx
- Tareas_U1_Autonomo_Análisis_Diseño_OO_PlantUML.docx